

iOS面试题合集（多线程篇）

iOSer iOSer 5月19日

收录于话题

#BAT大厂面试题合集

9个

欢迎点击上方蓝字“iOSer”关注我们
再点击右上角“...”菜单，选择“设为星标”
我们一起精进、成长！

本栏目将持续更新--请iOS的小伙伴关注我们!

做这个的初心是希望能巩固自己的基础知识，

当然也希望能帮助更多的开发者，

可以加入我们的技术交流群：**834688868**，群里每天都会有干货、技术动向、职业生涯、行业热点、职场趣事等一切有关于程序员的内容分享。

不管你是大牛还是小白都欢迎入驻，分享BAT，阿里面试题、面试经验，讨论技术，大家一起交流学习成长！

目录：

- 1.进程与线程分别是什么意思？
- 2.什么是多线程？
- 3.多线程的优点和缺点有哪些？
- 4.多线程的 并行 和 并发 有什么区别？
- 5.iOS中实现多线程的几种方案，各自有什么特点？
- 6.多个网络请求完成后如何执行下一步？
7. 多个网络请求顺序执行后如何执行下一步？
- 8.如何理解多线程中的死锁？
- 9.如何去理解GCD执行原理？

1.进程与线程分别是什么意思？

- 进程：

- 1.进程是一个具有一定独立功能的程序关于某次数据集合的一次运行活动，它是操作系统分配资源的基本单元.
- 2.进程是指在系统中正在运行的一个应用程序，就是一段程序的执行过程,我们可以理解为手机上的一个app.
- 3.每个进程之间是独立的，每个进程均运行在其专用且受保护的内存空间内，拥有独立运行所需的全部资源

- 线程

- 1.程序执行流的最小单元，线程是进程中的一个实体.
- 2.一个进程要想执行任务,必须至少有一条线程.应用程序启动的时候，系统会默认开启一条线程,也就是主线程

- 进程和线程的关系

- 1.线程是进程的执行单元，进程的所有任务都在线程中执行
- 2.线程是 CPU 分配资源和调度的最小单位
- 3.一个程序可以对应多个进程(多进程),一个进程中可有多个线程,但至少要有有一条线程
- 4.同一个进程内的线程共享进程资源

2.什么是多线程？

- 多线程的实现原理：事实上，同一时间内单核的CPU只能执行一个线程，多线程是CPU快速的在多个线程之间进行切换（调度），造成了多个线程同时执行的假象。

- 如果是多核CPU就真的可以同时处理多个线程了。
- 多线程的目的是为了同步完成多项任务，通过提高系统的资源利用率来提高系统的效率。

3.多线程的优点和缺点有哪些？

- 优点：

能适当提高程序的执行效率

能适当提高资源利用率（CPU、内存利用率）

- 缺点：

开启线程需要占用一定的内存空间（默认情况下，主线程占用1M，子线程占用512KB），如果开启大量的线程，会占用大量的内存空间，降低程序的性能

线程越多，CPU在调度线程上的开销就越大

程序设计更加复杂：比如线程之间的通信、多线程的数据共享

4.多线程的 并行 和 并发 有什么区别？

- 并行：充分利用计算机的多核，在多个线程上同步进行
- 并发：在一条线程上通过快速切换，让人感觉在同步进行

5.iOS中实现多线程的几种方案，各自有什么特点？

- NSThread 面向对象的，需要程序员手动创建线程，但不需要手动销毁。子线程间通信很难。
- GCD c语言，充分利用了设备的多核，自动管理线程生命周期。比NSOperation效率更高。
- NSOperation 基于gcd封装，更加面向对象，比gcd多了一些功能。

6.多个网络请求完成后如何执行下一步？

- 使用GCD的dispatch_group_t

创建一个dispatch_group_t

每次网络请求前先dispatch_group_enter,请求回调后再dispatch_group_leave，enter和leave必须配合使用，有几次enter就要有几次leave，否则group会一直存在。

当所有enter的block都leave后，会执行dispatch_group_notify的block。



```
NSString *str = @"http://xxxx.com/";
NSURL *url = [NSURL URLWithString:str];
NSURLRequest *request = [NSURLRequest requestWithURL:url];
NSURLSession *session = [NSURLSession sharedSession];
```

```
dispatch_group_t downloadGroup = dispatch_group_create();
for (int i=0; i<10; i++) {
    dispatch_group_enter(downloadGroup);
```

```
    NSURLSessionDataTask *task = [session dataTaskWithRequest:
    request completionHandler:^(NSData * _Nullable data, NSURLResp
    onse * _Nullable response, NSError * _Nullable error) {
        NSLog(@"%d---%d",i,i);
        dispatch_group_leave(downloadGroup);
```

```

    }];
    [task resume];
}

dispatch_group_notify(downloadGroup, dispatch_get_main_queue()
, ^{
    NSLog(@"end");
});

```

- 使用GCD的信号量dispatch_semaphore_t

dispatch_semaphore信号量为基于计数器的一种多线程同步机制。如果semaphore计数大于等于1，计数-1，返回，程序继续运行。如果计数为0，则等待。

dispatch_semaphore_signal(semaphore)为计数+1操作,dispatch_semaphore_wait(sema, DISPATCH_TIME_FOREVER)为设置等待时间，这里设置的等待时间是一直等待。

创建semaphore为0，等待，等10个网络请求都完成了，
dispatch_semaphore_signal(semaphore)为计数+1，然后计数-1返回



```

NSString *str = @"http://xxxx.com/";
NSURL *url = [NSURL URLWithString:str];
NSURLRequest *request = [NSURLRequest requestWithURL:url];
NSURLSession *session = [NSURLSession sharedSession];

dispatch_semaphore_t sem = dispatch_semaphore_create(0);
for (int i=0; i<10; i++) {

    NSURLSessionDataTask *task = [session dataTaskWithRequest:
request completionHandler:^(NSData * _Nullable data, NSURLResp
onse * _Nullable response, NSError * _Nullable error) {
        NSLog(@"%d---%d",i,i);
        count++;
        if (count==10) {
            dispatch_semaphore_signal(sem);
            count = 0;
        }
    }];
}
}];

```

```

        [task resume];
    }
    dispatch_semaphore_wait(sem, DISPATCH_TIME_FOREVER);

    dispatch_async(dispatch_get_main_queue(), ^{
        NSLog(@"end");
    });
}

```

7.多个网络请求顺序执行后如何执行下一步？

- 使用信号量semaphore

每一次遍历，都让其dispatch_semaphore_wait(sem, DISPATCH_TIME_FOREVER)，这个时候线程会等待，阻塞当前线程，直到dispatch_semaphore_signal(sem)调用之后



```

NSString *str = @"http://www.jianshu.com/p/6930f335adba";
NSURL *url = [NSURL URLWithString:str];
NSURLRequest *request = [NSURLRequest requestWithURL:url];
NSURLSession *session = [NSURLSession sharedSession];

dispatch_semaphore_t sem = dispatch_semaphore_create(0);
for (int i=0; i<10; i++) {

    NSURLSessionDataTask *task = [session dataTaskWithRequest:
    request completionHandler:^(NSData * _Nullable data, NSURLResp
    onse * _Nullable response, NSError * _Nullable error) {

        NSLog(@"%d---%d",i,i);
        dispatch_semaphore_signal(sem);
    }];

    [task resume];
    dispatch_semaphore_wait(sem, DISPATCH_TIME_FOREVER);
}

```

```
dispatch_async(dispatch_get_main_queue(), ^{
    NSLog(@"end");
});
```

8.如何理解多线程中的死锁？

死锁是由于多个线程（进程）在执行过程中，因为争夺资源而造成的互相等待现象，你可以理解为卡主了。产生死锁的必要条件有四个：

- 互斥条件：指进程对所分配到的资源进行排它性使用，即在一段时间内某资源只由一个进程占用。如果此时还有其它进程请求资源，则请求者只能等待，直至占有资源的进程用毕释放。
- 请求和保持条件：指进程已经保持至少一个资源，但又提出了新的资源请求，而该资源已被其它进程占有，此时请求进程阻塞，但又对自己已获得的其它资源保持不放。
- 不可剥夺条件：指进程已获得的资源，在未使用完之前，不能被剥夺，只能在使用完时由自己释放。
- 环路等待条件：指在发生死锁时，必然存在一个进程——资源的环形链，即进程集合{P0, P1, P2, ..., Pn}中的P0正在等待一个P1占用的资源；P1正在等待P2占用的资源，.....，Pn正在等待已被P0占用的资源。
- 最常见的就是 同步函数 + 主队列 的组合，本质是队列阻塞。



```
dispatch_sync(dispatch_get_main_queue(), ^{
    NSLog(@"2");
});

NSLog(@"1");
// 什么也不会打印，直接报错
```

9.如何去理解GCD执行原理？

- GCD有一个底层线程池，这个池中存放的是一个一个的线程。之所以称为“池”，很容易理解出这个“池”中的线程是可以重用的，当一段时间后这个线程没有被调用胡话，这个线程就会被销毁。注意：开多少条线程是由底层线程池决定的（线程建议控制再3~5条），池是系统自动来维护，不需要我们程序员来维护（看到这句话是不是很开心？）而我们程序员需要关心的是什么呢？我们只关心的是向队列中添加任务，队列调度即可。
- 如果队列中存放的是同步任务，则任务出队后，底层线程池中会提供一条线程供这个任务执行，任务执行完毕后这条线程再回到线程池。这样队列中的任务反复调度，因为是同步的，所以当我们用currentThread打印的时候，就是同一条线程。
- 如果队列中存放的是异步的任务，（注意异步可以开线程），当任务出队后，底层线程池会提供一个线程供任务执行，因为是异步执行，队列中的任务不需等待当前任务执行完毕就可以调度下一个任务，这时底层线程池中会再次提供一个线程供第二个任务执行，执行完毕后再回到底层线程池中。
- 这样就对线程完成一个复用，而不需要每一个任务执行都开启新的线程，也就从而节约的系统的开销，提高了效率。在iOS7.0的时候，使用GCD系统通常只能开58条线程，iOS8.0以后，系统可以开启很多条线程，但是实在开发应用中，建议开启线程条数：35条最为合理。

部分总结的多线程面试题就到这里了，
以上内容对你有帮助的话，记得关注我们！

文章来源于网络，已尽可能标明作者以及来源，文章内容为作者独立观点，不代表本公众号立场，因文章侵权本公

众号不承担任何法律及连带责任，如有侵权，请联系我们删除。



觉得不错的话，别忘了点个"在看"哦~ 📌

收录于话题 #BAT大厂面试题合集 · 9个

下一篇 · iOS面试题合集（网络篇）

[阅读原文](#)

喜欢此内容的人还喜欢

人均2300元，配齐一家三口保险！

深蓝保

被汽车撞伤，医保能报吗？这些情况，医保报不了

深蓝保

审查滴滴，释放了一个严厉的信号

21世纪商业评论